Group P03-03 - SRS

Introduction

Purpose

This document outlines the Software Requirements Specification for *EventHub – A Campus Event Management System*, Version 1.0. This SRS defines the functional and non-functional requirements for the system to be developed as part of a university software engineering project. The scope of this document includes the complete functionality expected in the final version of the *EventHub* system.

Document Conventions

Requirement Labels: Functional requirements are labeled as FR[X], and non-functional requirements as NFR[X] for easy cross-referencing.
User Story Labels: Similarly, User stories are labelled with US[X]
Priority Notation:
(Must) – Essential features for core functionality
(Should) – Important features, but not critical for initial deployment
(Optional) – Extension or nice-to-have features
Typography:
Bold is used for headings and key terms.
Code style is used for filenames, system variables, and user commands.
Inheritance of Priority: If not explicitly stated, detailed sub-requirements inherit the priority of the parent requirement.

Intended Audience and Reading Suggestions

This document is intended for the following stakeholders:
Developers – To understand functional and technical expectations for implementation.
Project Managers – To track project scope and validate milestone requirements.
Testers – To derive test cases from requirements.
Users and Club Organizers – To understand supported features and usability goals.
Documentation Writers – To prepare user manuals and help resources.
Suggested reading order:
Purpose & Product Scope – For all readers.
Overall Description – For developers and project managers.
Specific Requirements – For developers and testers.
Appendices & References – For support staff or further documentation.

Product Scope

EventHub is a comprehensive web-based platform designed to streamline the management, discovery, and participation in university campus events. The platform aims to empower student organizations to publish events and manage attendance while giving students a personalized, intuitive interface for browsing and RSVP-ing to events.

Objectives:
Simplify event creation and promotion for university clubs.
Provide personalized recommendations to improve student engagement.
Track event attendance and feedback.
Ensure real-time availability and locational data integration.

Benefits:
Centralized platform for all campus event activities.
Increased visibility and attendance for student-run events.
Improved administrative oversight with moderation tools.
This aligns with the university's goal to enhance campus life, student involvement, and digital innovation through modern, collaborative software solutions.

References

Wikipedia contributors. (n.d.). *IEEE 830-1998*. In *Wikipedia*. Retrieved August 7, 2025, fromhttps://en.wikipedia.org/wiki/Software_requirements_specification

EEE Standards Association. (1998). *IEEE Std 830-1998: Recommended practice for software requirements specifications*.
https://ieeexplore.ieee.org/document/720574

Wikipedia contributors. (n.d.). *Software requirements specification*. In *Wikipedia*. Retrieved August 8, 2025, from
https://en.wikipedia.org/wiki/Software_requirements_specification

Overall Description

Product Perspective

Event hub is a web-based application developed as a new system to support event management for RMIT University clubs and student organizations. It is not an extension or a replacement for existing products, but rather it is a self-contained platform that is built to meet specific needs of RMIT campus community. The system is designed to allow students to browse and RSVP to events, while giving the organizers the tools to create and manage event listings. This web application also supports admin users, with features for editing and use management.

Although it has even been integrated with external services such as email or cloud posting platforms, it does not form part of a larger system. A high-level architecture diagram outlining the system. Student and user interactions are included in the systems architecture section.

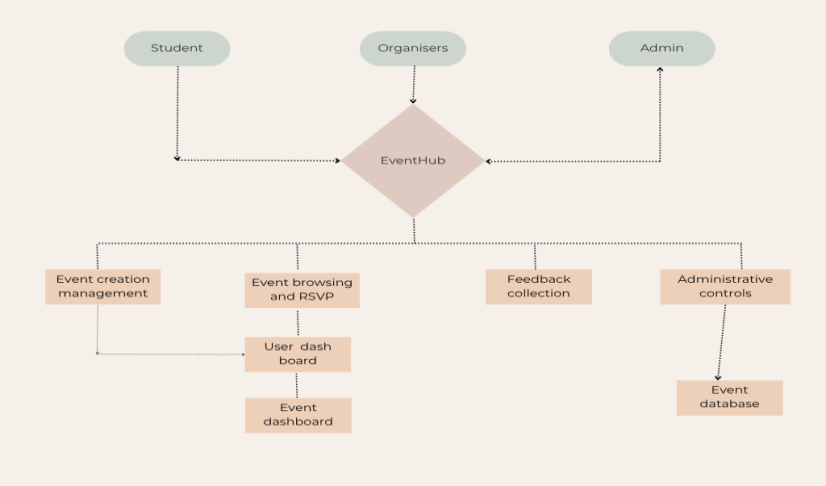
Product Functions

The Eventhub (A Campus Event Management System) is designed to help university clubs and students manage campus events efficiently by offering a set of key features that help and support both organizers and users.

To ensure the platform is simple to use and effective, Eventhub focuses on a few core functions that handle everything from creating and managing events to participation and administration.

- FR1 - Event creation and management – Organizers can create events by providing key details such as even title, description, date, time, location, and category. They can also edit or delete their own events as needed.
- FR2 - Event browsing and RSVP - students can explore upcoming events and filter them by category or keyword or RSVP to events that they are interested in attending.
- FR3 - Upcoming event recommendations - the system can suggest events to users based on the upcoming events
- FR4 - Feedback - after an event users can submit ratings and commence which helps improve future events and maintain event quality.
- FR5 - Administrative controls - Admins have access to moderation tools which can allow them to manage all events. Delete if there are any inappropriate content and handle user accounts. (suspending or banning users if necessary)
- FR6 - User Dashboard - each user has access to dashboard where they can view their RSVP history and receive upcoming event reminders

These functions work together to provide a user friendly and smooth experience for students and organizers, while giving administrators the control they need to keep the platform safe and reliable.



User Classes and Characteristics

Eventhub is intended to support three main types of users; Students, organizers and admins. Each group interacts with the event hub system in different ways based on the level of access and purpose for using the platform.

Students Class

- Frequency of use - Infrequent to frequent, based on personal interest in campus life
- Functions used - Browsing event calendars, RSVPing for events, giving feedback, and viewing their own personal dashboard
- Technical expertise - Basic computer and internet skills
- Privileges - may only view and engage with publicly available content
- Importance - Students represent the largest and most important user group to be satisfied, as the system is primarily there to enhance their campus experience

Organizer Class

- Frequency of use - Moderate to high, particularly during times of event planning
- Functions used - Creating, editing, and managing events; adding event photos; viewing RSVP lists
- Technical expertise - Basic computer and internet skills and comfortable managing listings and interpreting basic event feedback
- Privileges - Extended access to create and manage their own event content

- Importance - Organizers play a key role in keeping the platform active by regularly adding new events and engaging with attendees

Admin Class

- Frequency of use – High On demand, mainly for system management and moderation
- Functions used – User management, event acceptance or deletion, and content moderation
- Technical expertise - High computer and internet skills, capable of navigating all administrative tools and managing platform-wide settings
- Privileges – Full system access, including administrative configurations and sensitive data
- Importance - Although smaller in number, administrators are significant to maintaining the system intact, secure, and of sound overall quality

Operating Environment

EventHub is a web-based system that runs on current technology and software often found at universities or in the cloud.

The program will be created with Java 17 and the Spring Boot framework and will be able to operate on Windows, macOS, and Linux PCs. It will store data in a relational database, such as MySQL, which can be hosted locally or in the cloud. Docker will be utilised to easily package and deploy the application across several platforms.

Depending on the project requirements, the platform can be hosted on a local server or in the cloud. GitHub Actions will be used to automate testing and deployment as the project progresses.

Design and Implementation Constraints

(Note red sections indicate they are 'Yet to be decide' and are listed on appendix C)

1. *Regulatory / Ethical Constraints*
    a. *Must comply with Australian Privacy Act for user data storage and event content.*
    b. *Payment processing must meet Industry standards*
2. *Technology Constraints*
    a. *Frontend: Javalin / Thymeleaf to enable SEO-friendly pages that are responsive*

b. *Backend: Spring Boot –*
c. *Database: MySQL due to it's ability to support scalability*
d. *Must use RESTful API architecture for extensibility reasons*

3. *Security Constraints*
    a. *Role-based access control (Organizer, Admin, User) to prevent unauthorized actions (e.g., deleting events) from Users*
    b. *Use Spring Security to validate Users and give session tokens*

Integration Constraints

c. *Integration with Google Maps API for event location previews.*
d. *QR code check-in must integrate with supported scanners and mobile devices.*
e. *Public REST API must limit requests via API keys and respect rate limits.*

4. *Design Constraints*
    a. *Fully responsive UI for desktop, tablet, and mobile.*
    b. *Event pages must load in under 4 seconds for optimal user experience under average conditions*

Assumptions and Dependencies

Time Dependencies

The features of EventHub are divided into two categories: Core and extension features

Core features *(event creation, browsing/RSVP, feedback, user/admin dashboards)*

The core features must be implemented for the application to provide the intended value to students and organizers, and therefore will be prioritized in the initial development cycle.

Extension features *(Google Maps preview / API integration, QR check-in system, badge system, CSV export, public API).*

The implementation of extension features is dependent on project schedule and available development time. The final decision regarding the development of these features will be made at a later stage, once the core functionality has begun to be delivered and tested.

Hardware Dependencies

EventHub is designed to be a web-based system, so no specialized hardware is required just standard client devices running a web browser.

However, several of the ***extension features*** *rely on device functionality:*

- ***QR Check-In System*** *– Requires access to a camera (mobile or webcam) to scan the generated QR code.*
- ***Static Map Preview*** *– Depends on availability of internet and the client's ability to access the Google Maps API.*

Further, these hardware-related features will only function correctly if the client device and browser permits access.

External Dependencies

(Note red sections indicate they are ‘Yet to be decide’ and are listed on appendix C)

Some EventHub features rely on external APIs and services. Continued availability and correct operation of these services are assumed in order for the affected features to function.

- ***Google Maps API*** *– Used to display static previews of event locations on event detail pages. This is subject to rate-limiting and API key usage policies defined by Google.*
- ***SMTP Email Service*** *(e.g., Gmail SMTP) – No library or service has been decided but this will be necessary for email reminders*
- **QR Code Generation Library** – No library has been decided but this library will be needed to generate check-in codes for events.

If any of the above services change their terms of use or become unavailable, the related EventHub features may no longer function properly.

External Interface Requirements

User Interfaces

<Describe the logical characteristics of each interface between the software product and the users. This may include sample screen images, any GUI standards or product family style guides that are to be followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. Define the software components for which a user interface is needed. Details of the user interface design should be documented in a separate user interface specification.>

These decisions have not been finalised yet and will be updated laters – see appendix C

Hardware Interfaces

EventHub is a web-based application that does not require direct connection with any external hardware. The system is designed to work with ordinary computing hardware, such as personal PCs, laptops, and cloud-based virtual machines. These systems must support the necessary development and deployment technologies, such as Java 17, Docker, and a MySQL-compatible database. EventHub does not require any physical device integration, sensors, or embedded components, hence there are no extra hardware interface requirements beyond the server infrastructure utilised to host the program.

Software Interfaces

EventHub is built using a modular architecture based on the MVC (model view controller) pattern. It integrates with multiple external and internal software components. Below is a detailed overview of the interfaces and interactions.

Backend Framework

- **Spring Boot (v3.2+)**: Provides the core backend framework, managing HTTP requests, REST APIs, and the application lifecycle.
- **Maven (v3.9+)**: Build and dependency management tool used to resolve, and compile required libraries.

Database

- **MySQL (v8.0+)**: The relational database management system used to store persistent data, including user accounts, event details, RSVP records, and feedback.
- **JDBC (Java Database Connectivity)** is used for communication between the Spring Boot application and MySQL.
- Data items include:
    - Event: title, description, datetime, location, tags
    - User: name, email, password hash, role
    - RSVP: user ID, event ID, status
    - Feedback: event ID, rating, comment

Frontend

- HTML/CSS/JavaScript rendered from Spring Boot (Thymeleaf or REST API for frontend frameworks)
- Compatible with all modern browsers (see section 1.4 Operating Environment)

Other Integrated Components

- **JUnit5**: Used for writing and running unit and integration tests.
- **Docker (v24+)**: Used to containerize the backend and database for consistent deployment.

- **GitHub Actions**: Implements CI/CD pipelines for automated build, test, and deployment processes.

External APIs (Optional/Enhancements)

- **Google Maps API**: To display static previews of event locations.
- **Email Service (e.g., Gmail SMTP or Mailgun API)**: Sends RSVP confirmation and reminder emails.

Shared Data and Format

- All API responses and shared data structures use **JSON** format for REST interfaces.
- Database entities are shared using **JPA Entities** between service and persistence layers.
- Frontend forms send data using **application/x-www-form-urlencoded** or **application/json** content types.

Implementation Constraints

- All components must be Docker-compatible.
- JSON must be used for all inter-service communication.
- Configuration should be externalized using .env or application.yml.

Communications Interfaces

EventHub requires a range of communication mechanisms to ensure users and systems interact efficiently and securely.

Web Communication

- **HTTP/HTTPS Protocols** are used for:
  - All web traffic between browser and backend
  - REST API communication (in case of SPA frontend)
- **Ports Used**:
  - Port 80 for HTTP (development only)
- **Message Format**:
  - **Requests**: Form data or JSON (e.g., RSVP form, feedback submission)
  - **Responses**: JSON for APIs; HTML+CSS for server-rendered pages

Email Communication

- Used for:
  - RSVP confirmations
  - Event reminder notifications
  - Admin alerts (optional)
- **SMTP Protocol**: TLS-encrypted

- Integration via Spring Boot MailSender and environment-configured SMTP server

Security

- All data transferred between clients and servers must use **HTTPS with TLS 1.2+**
- Email credentials and API keys must be stored securely (e.g., via environment variables or Docker secrets)
- **JWT (JSON Web Tokens)** or **session-based authentication** will be used for secure access control.

Communication Standards

Protocol	Purpose
HTTP/HTTPS	General web communication
SMTP	Email services
REST	API endpoints
JSON	Data formatting for REST APIs
TLS 1.2+	Secure data transfer

Synchronization Mechanisms

- Database access is synchronized using Spring Data JPA and transactional annotations to prevent race conditions.
- All user sessions are stateless in REST mode or managed via HTTP sessions in MVC-based UI mode.
- Event-based email notifications are triggered using asynchronous tasks (@Async in Spring Boot).

Nonfunctional Requirements

1.2 Performance Requirements

- **NFR1: Concurrent Users**: The system must support at least 50 **concurrent users** without noticeable performance degradation.
- **NFR2: Response Time**: All user interactions (RSVP, event search, feedback submission) must receive a response within **2 seconds** under normal load.
- **NFR3: Event Search**: Filtering and searching events should return results in **under 1.5 seconds**.
- **NFR4: Gallery Upload**: Image uploads for event galleries should be completed in **< 5 seconds** for files up to 5MB.

- **NFR5: Database Query Time**: Queries for user RSVPs and event lists should execute in **< 200ms** on average.

2.2 Safety Requirements

- **NFR5: Data Protection**: EventHub must prevent data loss from unexpected failures by implementing **daily backups**.
- **NFR6: Input Validation**: All user inputs must be validated to avoid crashes, incorrect data storage, or UI failure.
- **NFR7: Deletion Confirmation**: Users must confirm before any destructive action (e.g., deleting an event).
- **NFR8: Fail-Safe Mechanism**: The system should log and gracefully handle any server or client-side exceptions without exposing sensitive error information.
- **NFR9: Image Upload Restrictions**: Limit file types and sizes to avoid security vulnerabilities and ensure platform stability.

3.2 Security Requirements

- **NFR10: Authentication**: Secure login using **username and password**, with **role-based access control** (Admin, Organizer, Student).
- **NFR11:Authorization**: Only authorized users (e.g., organizers or admins) can edit/delete events or manage user data.
- **NFR12: Encryption**: All sensitive data must be transmitted over **HTTPS using TLS 1.2 or higher**.
- **NFR13: Password Storage**: Use **bcrypt** to hash and salt passwords.
- **NFR14: Account Lockout**: Lock user accounts after **5 failed login attempts** to prevent brute-force attacks.
- **NFR15: Data Privacy**: Comply with **Australian Privacy Principles (APPs)** and RMIT's data handling policy.

4.2 Software Quality Attributes

Attribute	Specification
Usability	User-friendly interface; students should be able to RSVP to an event in <3 clicks.
Reliability	The app must have 99.5% uptime during university event hours (8 AM – 10 PM).
Maintainability	Follow **MVC architecture** using Spring Boot; modular codebase to support future enhancements.

Testability	Include **JUnit5 unit tests** for core functions; aim for **>80% test coverage**.
Availability	The system should recover from failures (e.g., crashes, power loss) within **5 minutes**.
Portability	The application must be usable on all modern browsers (Chrome, Firefox, Safari).
Accessibility	Meet **WCAG 2.1 Level AA** guidelines. Use semantic HTML, keyboard navigation, and alt text.
Extensibility	Designed to easily integrate extensions like payment gateway, QR check-in, and email notifications.

1.2 Business Rules

- **NFR16: Role Restrictions**:
- **NFR17:** Only **organizers** can create/edit/delete their own events.
- **NFR18:** Only **admins** can manage all events and users.
- **NFR19: RSVP Policy**: Students can RSVP to **multiple events but** cannot RSVP to the **same event twice**.
- **NFR20: Event Limits**: Events cannot exceed **a set max attendee count** defined by the organizer.
- **NFR21: Feedback Access**: Only users who attended an event (via RSVP) can submit feedback.
- **NFR22: Event Timing**: Events must be scheduled for **future dates only**; past-dated event creation is disallowed.
- **NFR23: Account Status**: Deactivated or banned users cannot RSVP or post new events.

2.2 Other Requirements

- **NFR24: Database**: Must use **MySQL** or equivalent relational DBMS (agreed by the tutor). Use foreign key constraints and indexing.
- **NFR25: CI/CD**: Use **GitHub Actions** for continuous integration; unit tests must pass before merging to main branch.
- **NFR26: Containerization**: Final deployment must use **Docker** and include Dockerfile and docker-compose.yml.
- **NFR27: Documentation**: All API endpoints and user flows must be documented and shared in the user guide.
- **NFR28: Legal**: EventHub must adhere to **RMIT's academic integrity and IT policies**, including licensing and ethical source code usage.

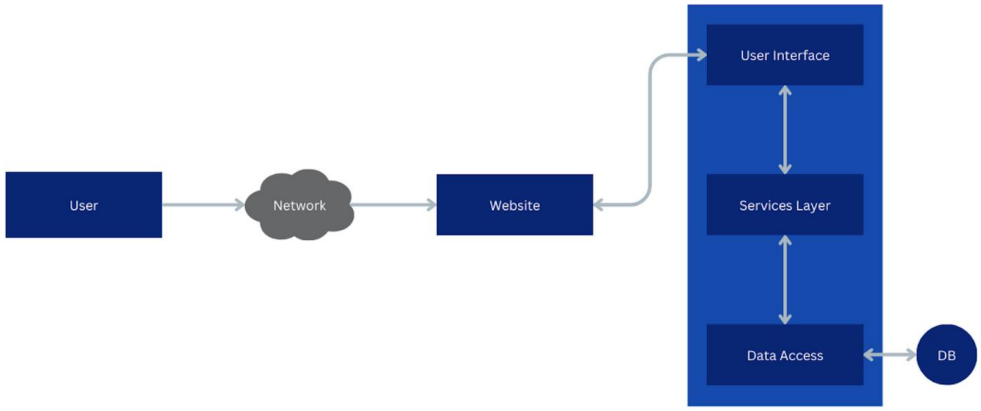
System Architecture

EventHub adopts a monolithic layered architecture pattern that consolidates all application components into a single deployable unit. The core decision implements a Spring Boot application that encompasses user interface, business logic, and data access layers within one executable. This monolithic approach provides structure for debugging efficiency whilst maintaining clear separation. The single deployment model aligns with the team's time constraints for the 12-week development timeline.

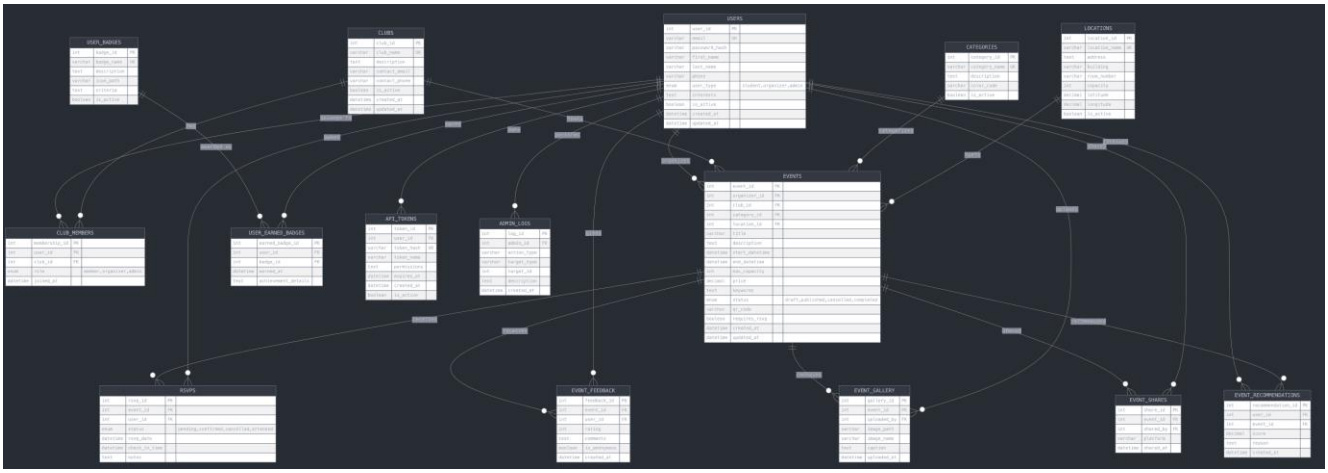
Spring Boot with Maven serves as the monolithic foundation, housing all application layers within a unified framework. AWS API Gateway and Lambda functions function as external data access services that the monolithic application calls through HTTP protocols, similar to how a monolith might interact with external databases or APIs. JavaScript handles client-side responsiveness and dynamic UI generation within the presentation layer. This technology integration maintains the single executable unit principle whilst using cloud services.

The monolithic architecture directly supports maintainability through simplified debugging and deployment processes inherent to single-unit applications. Spring Boot's environment ensures consistent development patterns and reduces complexity compared to distributed systems (microservices). Performance optimisation occurs within the monolithic boundaries through efficient internal component communication and Spring Boot's built-in optimisation features. The DevOps strategy using GitHub Actions supports the monolithic deployment model with streamlined build and deployment pipelines for the single application artefact, ensuring reliable delivery of the complete system as one cohesive unit.

Architecture Overview



Database Schema



Architectural Decisions

Title: ADR 1: Spring Boot as Primary Web Framework

Context: EventHub requires a robust web framework. The team needs a framework that supports rapid development, has extensive documentation, provides built-in security features, and integrates well with various databases. Spring Boot is specified in the requirements and is widely adopted in enterprise applications.

Decision: We will use Spring Boot 3.x as our primary web framework for developing the EventHub application.

Status: Accepted (Mandated by requirements)

Consequences:

- Positive: Extensive ecosystem, built-in security, easy database integration, large community support
- Positive: Auto-configuration reduces boilerplate code
- Negative: Steeper learning curve for team members unfamiliar with Spring
- Neutral: Requires Maven for dependency management

Title: ADR 2: MySQL as Primary Database

Context: The application requires persistent storage for events, users, RSVPs, and feedback. The system needs to handle relationships between entities (user-events, events-RSVPs). A relational database is recommended for the requirements. MySQL is free, widely supported, and has good Spring Boot integration.

Decision: We will use MySQL 8.0 as our primary relational database for all persistent data storage.

Status: Accepted

Consequences:

- Positive: ACID compliance ensures data integrity
- Positive: Excellent support for complex queries and relationships
- Positive: Good performance for read-heavy operations (event browsing)
- Negative: Requires careful indexing for search functionality
- Neutral: Team needs to manage database migrations

Title: ADR 3: GitHub Actions for Continuous Integration

Context: The assignment requires CI/CD implementation. GitHub Actions is integrated with our required GitHub repository, free for private repositories, and supports Docker containerization. The team is already using GitHub for version control.

Decision: We will use GitHub Actions for our CI/CD pipeline, running tests on every push and deploying them to a staging environment.

Status: Accepted

Consequences:

- Positive: Zero additional cost
- Positive: Integrated with our repository
- Positive: YAML-based configuration is version controlled
- Negative: Limited build minutes for private repositories
- Neutral: Team needs to learn GitHub Actions syntax

Title: ADR 5: Docker Containerization Strategy

Context: The assignment requires Docker implementation. Containerization ensures consistent environments across development, testing, and production. Docker Compose can orchestrate multiple services (web app, database).

Decision: We will containerize the application using Docker and use Docker Compose for local development with MySQL.

Status: Accepted (Mandated by requirements)

Consequences:

- Positive: Consistent environments across all stages
- Positive: Easy deployment and scaling
- Positive: Simplified onboarding for new developers
- Negative: Additional overhead for simple local development
- Neutral: Requires Docker knowledge from all team members

Title: ADR 4: Docker Containerization Strategy

Context: The assignment requires Docker implementation. Containerization ensures consistent environments across development, testing, and production. Docker Compose can orchestrate multiple services (web app, database).

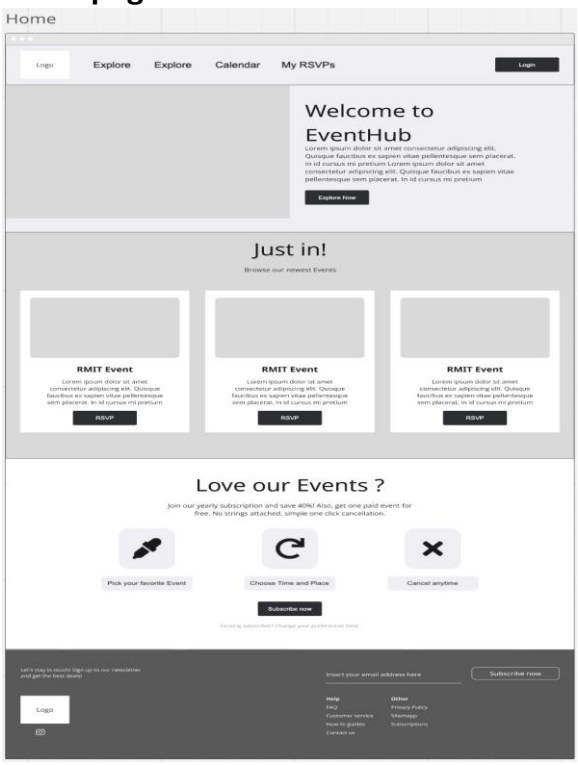
Decision: We will containerize the application using Docker and use Docker Compose for local development with MySQL.

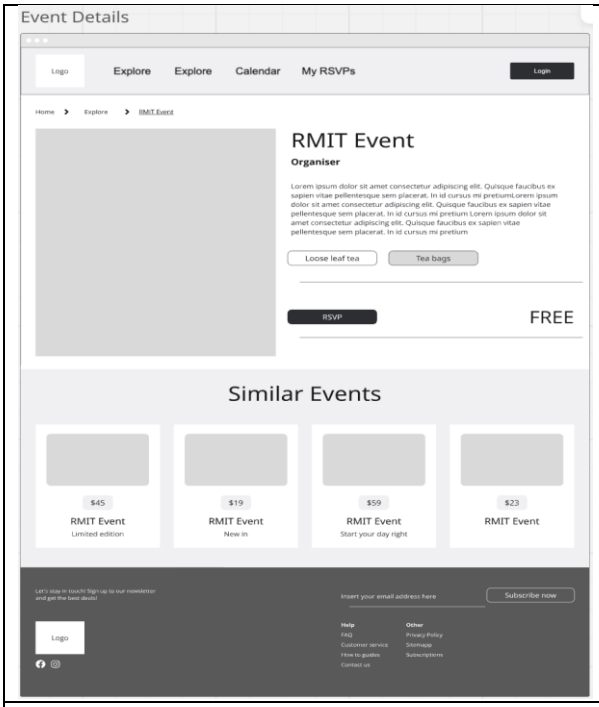
Status: Accepted (Mandated by requirements)

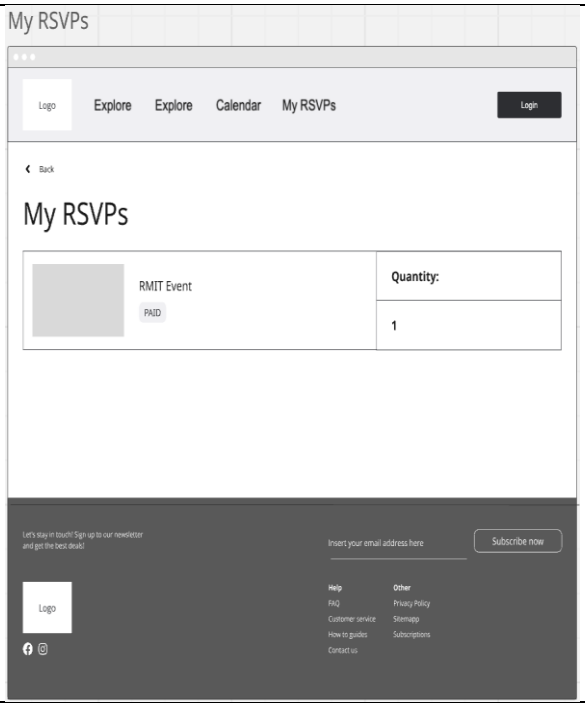
Consequences:

- Positive: Consistent environments across all stages
- Positive: Easy deployment and scaling
- Positive: Simplified onboarding for new developers
- Negative: Additional overhead for simple local development
- Neutral: Requires Docker knowledge from all team members

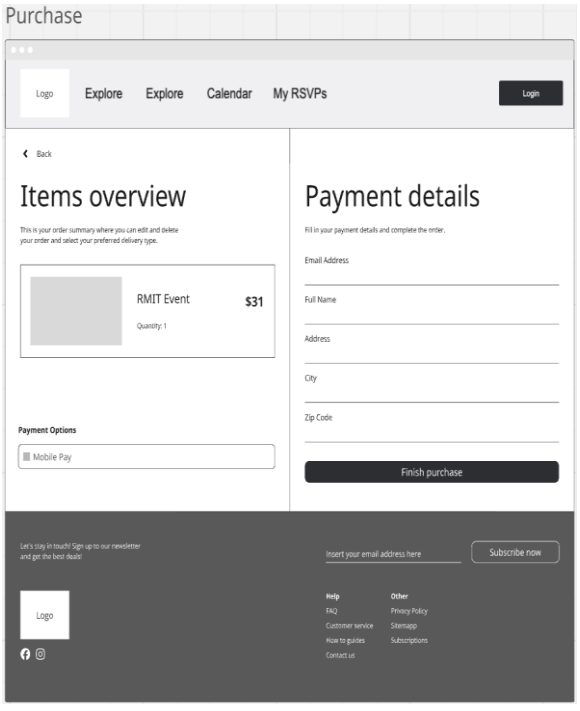
User Interface Design

Homepage:	**Explore:**
	
Event Details:	**RSVPs:**

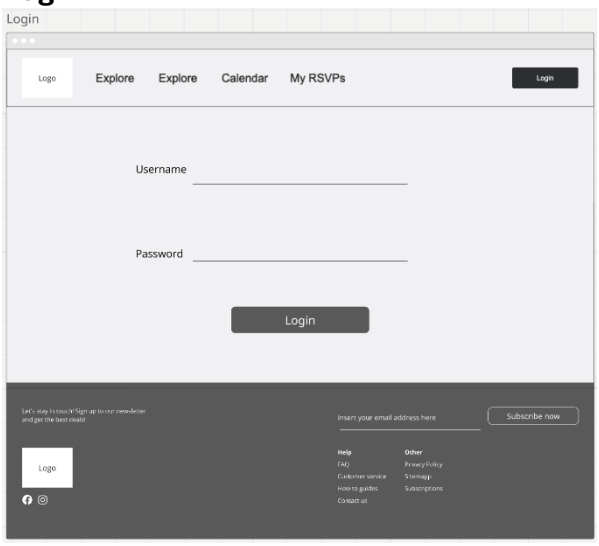




Purchase:

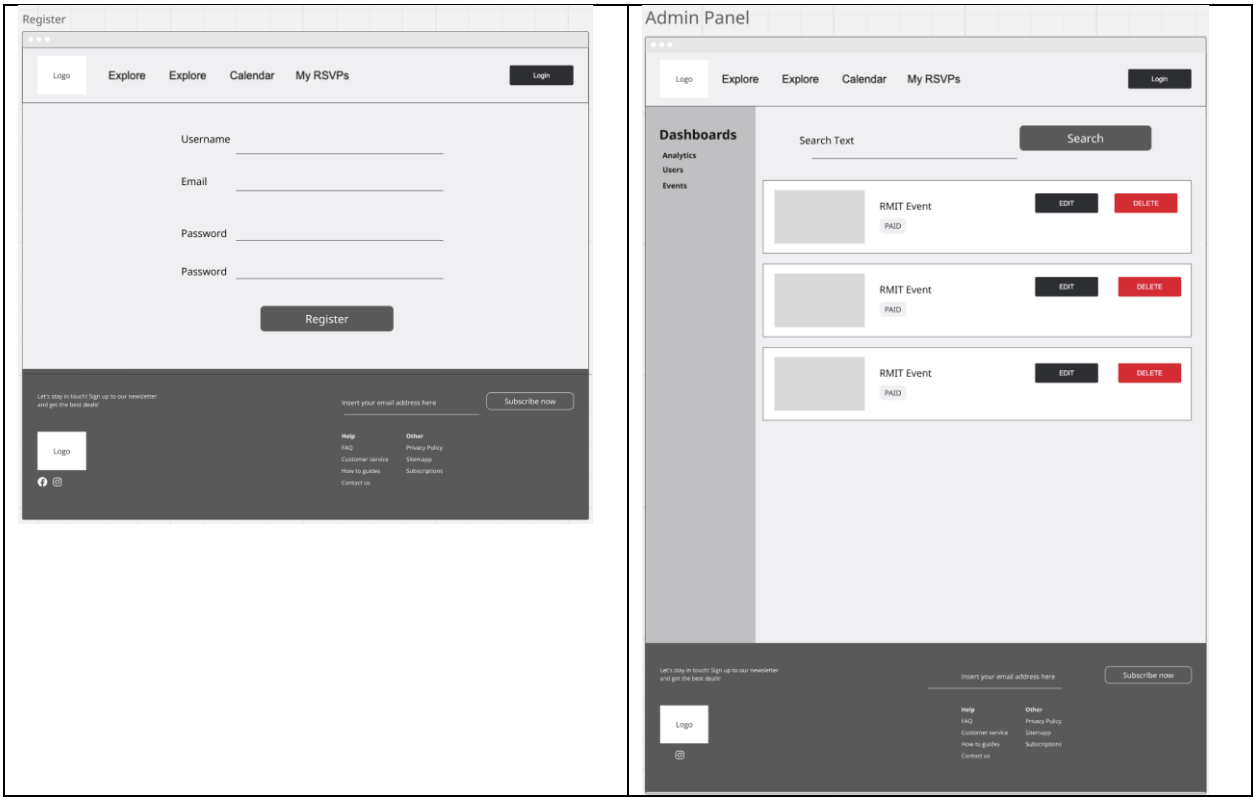


Login:



Register:

Admin Panel:



Appendix A: Glossary

Term	Definition (max. 30 words)
API (Application Programming Interface)	a set of rules that allows one application / component to communicate with another
REST API (Representational State Transfer)	an architectural style for designing web services that use standard HTTP methods.
QR Check-In	A feature that uses a QR code to allow users to check-in when attending an event.
GitHub Actions	A continuous integration/continuous deployment (CI/CD) service used to automate build and test workflows
Spring Boot	Java-based backend framework that is a requirement for this assignment /= specification
Thymeleaf	A Java template engine used mainly in Spring applications.
AWS	Amazon Web Services – cloud computing platform that provides on-demand services over the internet.

Docker	A tool that packages apps into portable containers for consistent, reliable deployment across environments.
Lambda	A serverless compute service by Amazon. It lets you run code without provisioning or managing servers.
UI / UX	User interface / User Experience (design and usability aspects of the system)
CI / CD	Continuous Integration / Continuous Delivery (or Deployment)
RSVP	*Répondez S'il Vous Plaît* (French: "Please Respond"), indicating attendance confirmation
CRUD	Create, Read, Update, Delete
SRS	Software Requirements Specification
SQL	Structured Query Language
ID	Identifier (unique reference for records)
HTTPS	HyperText Transfer Protocol Secure (not used locally but included in document for scalability)
HTTP	HyperText Transfer Protocol (used locally for this project)
JSON	JavaScript Object Notation (commonly used data format for APIs)
ERD	Entity Relationship Diagram (used in database design)
IDE	Integrated Development Environment
MVC	Model-View-Controller (common web app architecture pattern)
QA	Quality Assurance (testing process)
CRUD Ops	Shorthand for CRUD Operations (applied to Events, Users, etc.)

Appendix B: Analysis Models

USER STORIES

US 1 – Login

As an event-goer
I want to sign up and log in securely
So that I can manage tickets safely

Acceptance Criteria:
GIVEN a registered event-goer has created an account
WHEN they enter their valid username/email and password on the login page
THEN the system should authenticate them using the configured authentication library
AND they should have access to the web application

US 2 – Event Creation

As an organiser
I want to create a new event with title, description, date/time, location, and tags
So that students can see and RSVP to it.

Acceptance Criteria:
GIVEN the organiser is logged in
WHEN they fill in all required fields (title, description, date/time, location)
AND optionally add tags
AND click “Create Event”
THEN the event is saved in the database
AND it appears in the student event list

US 3 – RSVP Event

As a student, I would like to RSVP to the event so that I can participate in the event.
Acceptance Criteria:
RSVP to an event:
Given I am a logged-in student
When I click the RSVP button on an event
Then my RSVP should be recorded
And I should not be able to RSVP to the same event more than
once
Cancel an RSVP:
Given I am a logged-in student who has RSVP’d to an event
When I choose to cancel my RSVP before the event starts
Then my RSVP should be removed from the system
And I should no longer appear in the RSVP list for that event

US 4 – Search and Filtering Events

As a student,
I want to search and filter events by keywords, dates or category,
so i can find events that match my interests easily.

Acceptance Criteria:

GIVEN a user is on the events page with access to the search bar
WHEN they type a keyword such as part of the event title or description
THEN the search results should dynamically update to display only upcoming events
AND if no events match the query, a clear and self-explanatory "No events found" message should be displayed

US 5 – Event Reminders

As a student,
I want to receive event reminders
so I don't forget to attend.

Acceptance Criteria:
GIVEN the student has RSVP'd to an event
WHEN the event date is approaching (e.g., 24 hours before start)
THEN the system sends a reminder notification
AND the reminder includes the event's title, date/time, and location

US 6 – Feedback Features
As a participant, I would like to leave my thoughts after attending the event so that the future event can be improved.

Acceptance Criteria:
Submitting feedback after attending an event:
Given I am a user who RSVP'd and attended the event
When I open the feedback form for that event
Then I must provide a rating between 1 and 5
And I may optionally provide a text comment

And I can only submit feedback once for the same event
And after I submit, I should see a confirmation message saying "Thank you for your feedback"

Organizer viewing aggregated feedback:
Given I am an organizer of an event
When I view the event feedback page
Then I should see the average rating for my event
And I should see all participant comments

US 7 – Event Deletion

As an administrator,
I want to delete inappropriate or misleading events,
So that new visitors aren't confused.

Acceptance Criteria:
GIVEN an administrator is logged into the system with valid admin privileges
WHEN they click the delete button for an event in the admin UI
THEN a confirmation prompt should appear asking them to confirm the deletion
AND once confirmed, the system should call the authenticated event deletion API endpoint
AND the system should log the deletion action and remove the event from all user views

US 8 – Event Sharing

As an event-goer,
I want to be able to share events,
so that I can attend with friends

Acceptance Criteria:
GIVEN an event-goal is logged into the system and viewing an event detail page
WHEN they click the “Share” button on the event detail page UI
THEN the system should generate a copyable event link unique to that event
AND the user should be able to copy and paste the link into messages or social platforms

AND the system should log the deletion action and remove the event from all user views

US 9 – View Event Detail
As a user, I would like to view the full details of the event, so that I know if the event interests me.
Acceptance Criteria:
Given I am a user on the event list page
When I select an event from the list
Then the system shall display the event details page
And the page shall include the event title
And the page shall include the event description
And the page shall include the event date and time
And the page shall include the event location
And the page shall include the organizer name
And the page shall include the category or tags
And if the event has an image gallery, the images should be visible
And if the event has a maximum capacity, the current availability should be displayed
And I should be able to navigate back to the event list from the details page

US 10 – Event Gallery

As an event organizer
I want to upload and display event photos after the event
So that attendees can view them on the event detail page
Acceptance Criteria:

GIVEN the organizer is logged into their dashboard and selects an event they own
WHEN they access the “Upload Photos” section and upload multiple images with preview
THEN the system stores each image linked to the correct event ID
AND the uploaded images are displayed in an organized gallery layout on the event detail page
AND the organizer can delete or replace photos from the gallery at any time

US 11 - Calendar View

As a student
I want to view events in a monthly or weekly calendar format
So that I can plan my schedule more easily

Acceptance Criteria:

GIVEN the user is on the calendar page
WHEN they view events in either month or week view
THEN events are displayed on their respective dates
AND the current date is highlighted
AND filters can be applied to view events by category or keyword
AND clicking on an event opens its event detail page

US 12 – Profile Management

As a student,
I want to manage my profile (name, email, profile picture, and interests)
So that event recommendations and RSVPs are personalised.

Acceptance Criteria:
GIVEN the student has not RSVP'd to an event
WHEN the event date is approaching
THEN no reminder notification is sent

US13 - Edit event details

As an organizer, I want to edit event details so that I can update changes in schedule or correct typos/mistakes

Acceptance Criteria:

GIVEN an event exists in the system
AND the logged-in user is either the event creator or an administrator
WHEN they choose to edit the event details and submit changes

THEN the system should update the event with the new details
AND display a confirmation message affirming that the change was intentional

US14 - Export attendee list

As an organizer, I want to export attendee list to csv so that I can record statistics and make it into a report

Acceptance Criteria:

GIVEN an event exists with attendees
AND the logged-in user is either the event organizer or an administrator
WHEN they choose to export attendee data
THEN the system should generate a CSV file
AND the file must include each attendee's name, email, check-in status, and RSVP status
AND the exported file should download correctly as a `.csv`

US15 – Event Check-in

As an event organiser, I want to track real-time attendance at my events so that I can monitor turnout and manage capacity.

Acceptance Criteria:

GIVEN An event has an active QR code and the attendee is registered

WHEN the attendee scans the QR code at the event location

THEN their attendance is recorded in the system

AND they receive confirmation of successful check-in

AND their check-in timestamp is logged for the event planner

US16 – Admin Dashboard

As an admin, I want to manage user accounts (deactivate/ban users) so that I can enforce community guidelines and handle problematic behaviour.

Acceptance Criteria:

GIVEN An admin is viewing the user management section

WHEN they select a user account and choose to deactivate it

THEN the user account is marked as deactivated

AND the user loses access to the platform

AND the action is logged with timestamp and admin ID

US16 – Google Maps Intergration

As a student, I want to see event locations on a map preview so that I can easily find venue locations and plan my route.

Acceptance Criteria:

GIVEN An admin is viewing the user management section

WHEN they select a user account and choose to deactivate it

THEN the user account is marked as deactivated

AND the user loses access to the platform

AND the action is logged with timestamp and admin ID

User Documentation - Section under maintenance as user documentation is decided

1. Email and contact information will be in the website footer in the help section
2. Contactable through GitHub
3. User Manuals
4.

DEFINITION-OF-DONE

A user story, task, or feature is considered Done when:

- **Code Quality & Testing**
  - All GitHub Actions (build, unit tests, linting, etc.) pass successfully.
  - Code has been peer-reviewed, with feedback addressed.
  - Unit and integration tests cover agreed functionality with sufficient test coverage.
- **Acceptance Criteria**
  - All acceptance criteria defined in the user story are met.
  - Edge cases and error handling have been considered and tested.
- **Documentation**
  - Relevant documentation (code comments, README, API docs, user guides) is updated where necessary.
- **Version Control**
  - Pull request has been approved, finalized, and merged into the main branch.
- **Completeness**
  - All required functionality is implemented.
  - Testing has been performed
  - No critical bugs or blockers remain open.

Appendix C: To Be Determined List

5. See Definition of Done in the repository: docs/Definition of done.docx - is always being updated

Technology/library decisions

6. Email service / library for email notifications
7. QR code generation Library